Lab 1 - Memory Hierarchy in gem5

Zhang Ruida, Hugo Bueno
Embedded Processing Architectures

1. Results

1.1 Result

The structure we built in gem5 is shown in Figure 1, including the CPU and L1-level data cache and instruction cache L2 cache multi-level cache design, and the data design and change according to Figure 2.

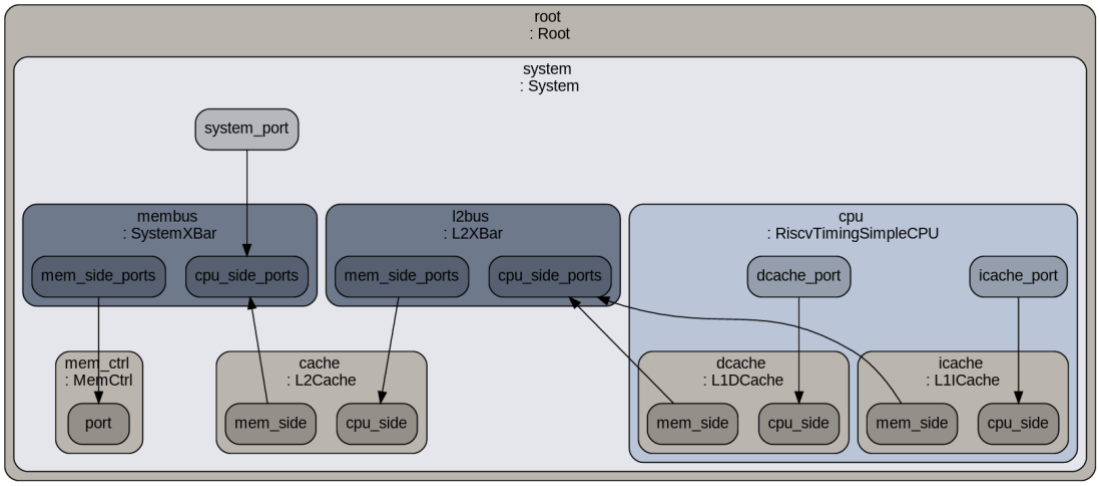

Figure 1 Block diagram of the system to be implemented in the project

Component	Parameter	Value(s)
Cache Block	Size	64 B
L2 Cache	Size	256 KiB
	Organization	8-way SA
	Replacement	LRU
L1 ICache	Size	4 KiB
	Organization	2-way SA
	Replacement	Random
L1 DCache	Size	256 B, 1 KiB, 4 KiB
	Organization	DM, 2-way SA, 4-way SA, FA
	Replacement	LRU
Matrices	Size	8×8, 64×64

Figure 2 System Configuration

1.2 Execution time

Table 1 Matrices 8*8 Execution Time

Execution Time(ms)	DM	2-way SA	4-way SA	FA
256B	0.143	0.105	0.091	0.091
1KiB	0.093	0.086	0.083	0.082
4KiB	0.082	0.082	0.082	0.082

Table 2 Matrices 64*64 Execution Time

Execution Time(ms)	DM	2-way SA	4-way SA	FA
256B	71.149	52.767	45.923	45.923
1KiB	54.056	47.114	45.562	45.922
4KiB	49.706	45.923	45.561	45.535

A 64 by 64 matrix takes more time to compute than an 8 by 8 matrix, which decreases as the size of the L1 data cache increases. Increasing the relevance of the L1 data cache usually reduces the computation time by reducing cache collisions and improving the hit ratio.

Increasing the size of the L1 data cache can potentially reduce computation time due to a decrease in cache misses. When the cache size is larger, it can hold more data and instructions, reducing the need to fetch data from slower memory levels.

However, excessively increasing the cache size might also lead to increased access latency due to longer lookup times and potentially increased contention among cache lines.

Increasing the associativity of the L1 data cache generally reduces computation time by decreasing cache conflicts and improving hit rates.

Moving from direct-mapped to set-associative and fully associative designs allows for more flexibility in cache line placement, reducing the likelihood of conflicts and subsequent cache

1.3 Local miss rate (L1 data cache)

Table 3 8*8 Local miss rate (L1 data cache)

Local miss rate (L1 data cache)	DM	2-way SA	4-way SA	FA
256B	0.354623	0.320875	0.311071	0.311071
1KiB	0.20031	0.16987	0.171142	0.156056
4KiB	0.092479	0.041181	0.038233	0.005571

Table 4 64*64 Local miss rate (L1 data cache)

Local miss rate (L1 data cache)	DM	2-way SA	4-way SA	FA
256B	0.160909	0.100772	0.05824	0.05824
1KiB	0.092115	0.054542	0.042571	0.044378
4KiB	0.064767	0.043321	0.040707	0.04015

The local miss rate refers to the proportion of cache accesses that result in a miss within the specific cache level (L1 data cache in this case).

Increasing the cache size generally reduces the local miss rate because a larger cache can accommodate more data, reducing the likelihood of cache evictions and subsequent misses.

Therefore, as the size of the L1 data cache increases from 256B to 1KiB to 4KiB, the local miss rate decrease.

Therefore, transitioning from DM to 2-way SA to 4-way SA to FA should progressively decrease the local miss rate.

The local miss rate becomes more important for larger matrix sizes, as the amount of data exceeds the capacity of the cache. Changes in cache organization, such as increasing associativity, can help reduce the local miss rate by accommodating more data in the cache and mitigating conflict misses.

1.4 Global miss rate (L1 data cache)

Table 5 8*8 Global miss rate (L1 data cache)

global miss rate (L1 data cache)	DM	2-way SA	4-way SA	FA
256B	0.078201945	0.071281344	0.069249805	0.069249805
1KiB	0.045742369	0.039075295	0.039351852	0.036013755
4KiB	0.021659826	0.00976241	0.009069952	0.001331852

Table 6 64*64 Global miss rate (L1 data cache)

global miss rate (L1 data cache)	DM	2-way SA	4-way SA	FA
256B	0.048727763	0.031082501	0.018200798	0.018200798
1KiB	0.028494997	0.017076445	0.013379083	0.01393916
4KiB	0.020213858	0.013611515	0.012800834	0.01262794

The global miss rate considers cache misses across all levels of the cache hierarchy.

Changing the organization of the L1 data cache can influence the global miss rate by affecting how misses propagate to higher-level caches.

Higher associativity typically reduces global miss rates by mitigating both conflict and compulsory misses.

However, other factors such as memory access patterns and cache replacement policies also play a role in determining the global miss rate.

The global miss rate may also benefit from changes in cache size and organization, but the impact may be less significant compared to larger matrix sizes. With a smaller dataset, the likelihood of accessing the same data multiple times is reduced, potentially limiting the benefits of larger caches.

1.5 Local/Global miss rate (L2 cache)

Table 7 8*8 Local miss rate (L2 cache)

Local miss rate (L2 data cache)	DM	2-way SA	4-way SA	FA
256B	0.02217	0.024691	0.025349	0.099549
1KiB	0.036354	0.0415	0.04135	0.044261
4KiB	0.063386	0.097291	0.100376	0.154787

Table 8 64*64 Local miss rate (L2 cache)

Local miss rate (L2 data cache)	DM	2-way SA	4-way SA	FA
256B	0.001395	0.002209	0.003725	0.003725
1KiB	0.00243	0.004129	0.005276	0.005064
4KiB	0.003485	0.005191	0.005517	0.005599

Table 9 8*8 Global miss rate (L2 cache)

global miss rate (L2 data cache)	DM	2-way SA	4-way SA	FA
256B	0.00201719	0.00207505	0.00208542	0.00208542
1KiB	0.00213067	0.00214627	0.00215398	0.00215322
4KiB	0.00218529	0.00221187	0.00221341	0.0022307

Table 10 64*64 Global miss rate (L2 cache)

global miss rate (L2 data cache)	DM	2-way SA	4-way SA	FA
256B	6.9082E-05	7.0456E-05	7.1291E-05	7.1291E-05
1KiB	7.0661E-05	7.1564E-05	7.1695E-05	7.1654E-05

4KiB	7.1198E-05	7.1818E-05	7.1784E-05	7.189E-05

1.6 Miss penalty (L1 data cache)

Table 11 8*8 Miss penalty (L1 data cache)

Miss penalty (L1 data cahce)	DM	2-way SA	4-way SA	FA
256B	825131000	710917000	675752000	675752000
1KiB	460697000	378658000	382122000	355004000
4KiB	217706000	109320000	103395000	36042000

Table 12 64*64 Miss penalty (L1 data cache)

Miss penalty (L1 data cache)	DM	2-way SA	4-way SA	FA
256B	37549396000	17671396000	10215869000	10215869000
1KiB	18637191000	9549577000	7469935000	7789179000
4KiB	11900316000	7600186000	7145384000	7050329000

Miss penalty refers to the additional time required to fetch data from a slower memory level (e.g., L2 cache or main memory) in the event of a cache miss.

Increasing the L1 data cache size may decrease the miss penalty because a larger cache can store more data, reducing the frequency of accessing slower memory levels.

However, the miss penalty is also affected by factors such as cache access latency and memory subsystem characteristics.

Increasing associativity can reduce miss penalties by mitigating cache conflicts and improving hit rates.

However, fully associative caches may have slightly higher access latencies due to more complex cache lookup operations.

Therefore, the impact on miss penalties will depend on the balance between reduced misses and increased access latency.

Miss penalties for larger matrix sizes are typically higher due to the increased likelihood of cache misses and longer memory access latencies. Improvements in cache size and organization can help mitigate miss penalties by reducing the frequency of cache misses and improving cache hit rates.

1.7 Miss penalty (L2 cache)

Table 13 8*8 Miss penalty (L2 cache)

Miss penalty (L2 data cahce)	DM	2-way SA	4-way SA	FA
256B	71791000	74212000	73912000	73912000
1KiB	73531000	73873000	74707000	74207000
4KiB	73508000	74007000	73726000	74098000

Table 14 64*64 Miss penalty (L2 cache)

Miss penalty (L2 data cache)	DM	2-way SA	4-way SA	FA
256B	158183000	158083000	158704000	158704000
1KiB	159687000	161409000	156298000	157288000
4KiB	158933000	158378000	156048000	157109000

2. Modify the baseline matrix multiplication algorithm

2.1 Code

```
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <inttypes.h>
#include <sys/time.h>

#define BLOCK_SIZE 32 // Adjust the block size as needed

void matmul_block(size_t size, uint32_t *a, uint32_t *b, uint32_t *c) {
    for (int jj = 0; jj < size; jj += BLOCK_SIZE) {
        for (int kk = 0; kk < size; kk += BLOCK_SIZE) {
            for (int i = 0; i < size; i++) {
                for (int j = jj; j < jj + BLOCK_SIZE && j < size; j++) {
                    uint32_t sum = 0;
                    for (int k = kk; k < kk + BLOCK_SIZE && k < size; k++) {
                        sum += a[i * size + k] * b[k * size + j];
                    }
                    c[i * size + j] += sum;
```

```
            }
        }
    }
}

int main(int argc, char *argv[]) {
    int i, j;
    size_t size;
    uint32_t *a = NULL, *b = NULL, *c = NULL;
    struct timeval t0, tf;
    float t;

    size = (argc > 1) ? ((atoi(argv[1]) > 0) ? atoi(argv[1]) : 1024) : 1024;
    printf("Matrix multiplication (%zux%zu)\n", size, size);

    a = malloc(size * size * sizeof(uint32_t));
    b = malloc(size * size * sizeof(uint32_t));
    c = calloc(size * size, sizeof(uint32_t)); // Initialize result matrix with zeros

    if (a == NULL || b == NULL || c == NULL) {
        printf("Memory allocation failed.\n");
        return 1;
    }

    // Initialize matrices
    for (i = 0; i < size; i++) {
        for (j = 0; j < size; j++) {
            a[i*size + j] = rand();
            b[i*size + j] = rand();
```

```
        }
    }

    gettimeofday(&t0, NULL);
    matmul_block(size, a, b, c);
    gettimeofday(&tf, NULL);

    t = ((tf.tv_sec - t0.tv_sec) * 1000.0) + ((tf.tv_usec - t0.tv_usec) / 1000.0);
    printf("Execution time: %.3f ms\n", t);

    // Free allocated memory
    free(a);
    free(b);
    free(c);

    return 0;
}
```

2.2 Proposed Modification

To exploit data locality further, we can utilize block matrix multiplication. This involves breaking down the matrices into smaller blocks and performing multiplication on these blocks. This approach enhances cache utilization and reduces memory accesses.

2.3 Result

The worst result is to choose 256B size and DM organization when the size of matrix is 64*64. The execution time is 71.149ms. After using modified algorithm, the execution time is down to 53.789ms